

Unit-4: Model Diagnosis and Tuning

4.1 Bias and variance in machine learning

- Bias and variance are fundamental concepts in machine learning that help us to understand model performance and generalization.
- If the machine learning model is not accurate, it can make predictions errors and these prediction errors are usually known as Bias and Variance.
- In machine learning, these errors will always be present as there is always a slight difference between the model predictions and actual predictions.
- The main aim of ML/data science analysts is to reduce these errors in order to get more accurate results.

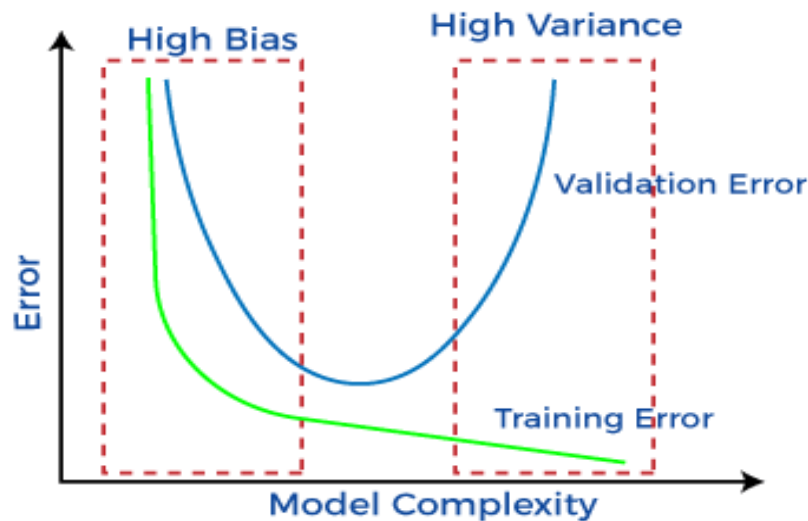


Fig: Bias and variance in machine learning

Bias vs. Variance

Bias - The error introduced due to simplifying assumptions in the learning algorithm.

A high-bias model tends to:

- Underfit the data (fails to capture patterns).

- Make strong assumptions, leading to inaccuracies.
- Examples: Linear regression, decision trees with few splits.

Variance - The error introduced due to sensitivity to small fluctuations in the training data. A high-variance model tends to:

- Overfit the data (memorizes patterns without generalizing).
- Be too complex, leading to poor performance on unseen data.
- Examples: Deep neural networks, complex decision trees.

The Bias-Variance Tradeoff

- If bias is too high, the model is overly simplified and underfits the data.
- If variance is too high, the model is overly complex and overfits the data.
- The ideal model balances bias and variance to achieve good generalization.

Real-life example: Imagine predicting house prices:

High-bias model: Uses only one feature (e.g., square footage), ignoring important factors like location and amenities.

High-variance model: Uses too many features and learns noise, making predictions inconsistent.

Reducing bias and variance

- Increase model complexity (Reduce bias): Use more powerful models, add features.
- Use regularization (L1, L2 penalties) (Reduce variance): Prevent the model from becoming too complex.
- Cross-Validation: Helps find the right balance by testing on different data subsets.
- Ensemble methods (Bagging, Boosting): Combine multiple models to reduce variance and improve accuracy.

Errors in machine learning - In machine learning, an error is a measure of how accurately an algorithm can make predictions for the previously unknown dataset. On the basis of these errors, the machine learning model is selected that can perform best on the particular dataset. There are mainly two types of errors in machine learning, which are:

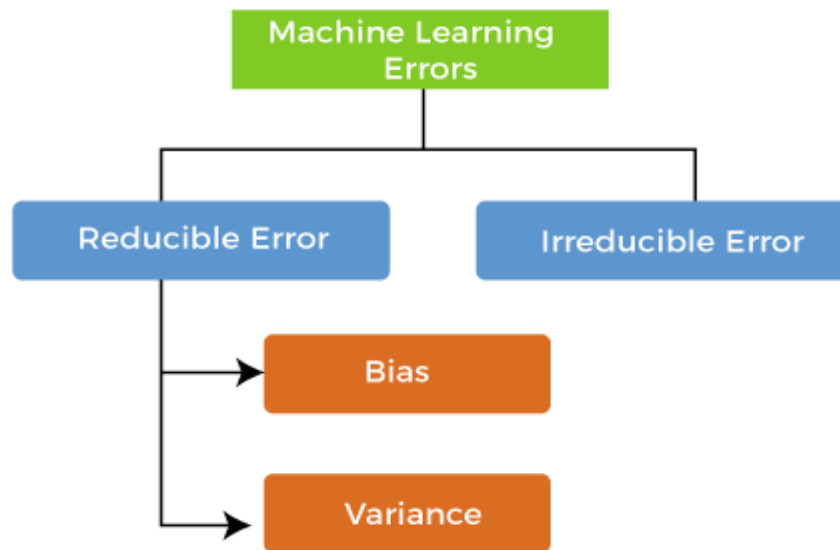
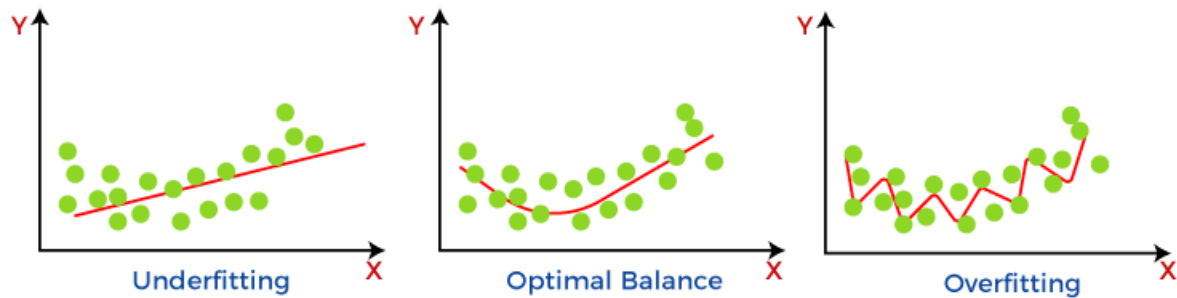


Fig: Errors in machine learning

- **Reducible errors:** These errors can be reduced to improve the model accuracy. Such errors can further be classified into bias and Variance.
- **Irreducible errors:** These errors will always be present in the model regardless of which algorithm has been used. The cause of these errors is unknown variables whose value can't be reduced.



Bias-Variance Trade-Off

While building the machine learning model, it is really important to take care of bias and variance in order to avoid overfitting and under fitting in the model. If the model is very simple with fewer parameters, it may have low variance and high bias. Whereas, if the model has a large number of parameters, it will have high variance and low bias. So, it is required to make a balance between bias and variance errors, and this balance between the bias error and variance error is known as **the Bias-Variance trade-off**.

For an accurate prediction of the model, algorithms need a low variance and low bias. But this is not possible because bias and variance are related to each other:

- If we decrease the variance, it will increase the bias.
- If we decrease the bias, it will increase the variance.

Bias-Variance trade-off is a central issue in supervised learning. Ideally, we need a model that accurately captures the regularities in training data and simultaneously generalizes well with the unseen dataset. Unfortunately, doing this is not possible simultaneously. Because a high variance algorithm may perform well with training data, but it may lead to overfitting to noisy data. Whereas, high bias algorithm generates a much simple model that may not even capture important regularities in the data. So, we need to find a sweet spot between bias and variance to make an

optimal model. Hence, the Bias-Variance trade-off is about finding the sweet spot to make a balance between bias and variance errors.

4.2 K-Fold Cross-Validation

- K-fold cross-validation is a powerful technique used to evaluate machine learning models.
- K-fold cross-validation is a technique for validating the model efficiency by training it on the subset of input data and testing on previously unseen subset of the input data.
- K-fold cross-validation approach divides the input dataset into K groups of samples of equal sizes.
- These samples are called **folds**. For each learning set, the prediction function uses k-1 folds and the rest of the folds are used for the test set.

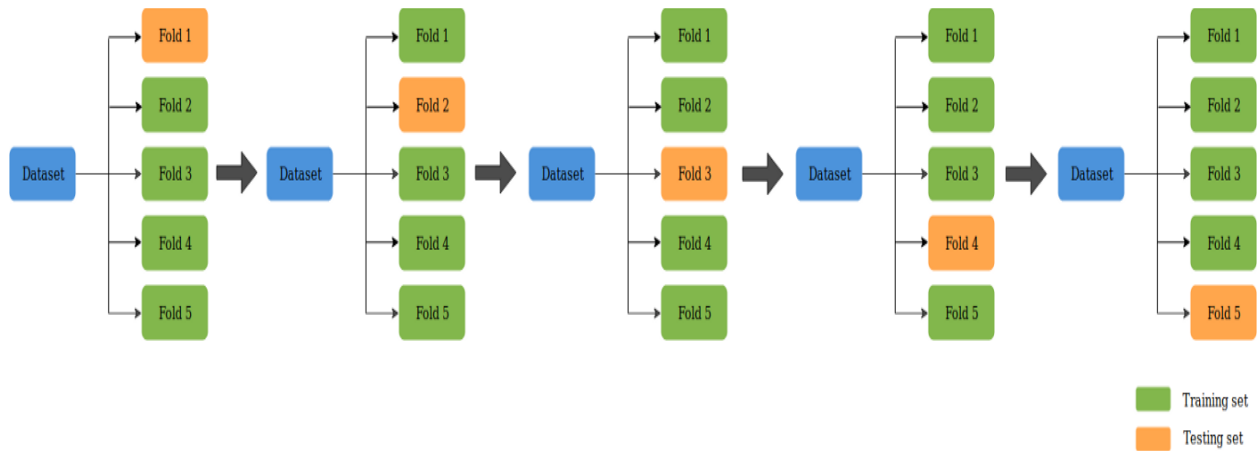
The steps for k-fold cross-validation are:

- Split the input dataset into K groups
- For each group:
 - Use K-1 folds for training.
 - Use the remaining fold for testing.
 - Repeat the process K times, changing the test fold each time.
 - Calculate the performance metric (e.g., accuracy, RMSE) for each of the K runs.
- Average the results to get the final performance estimate.

For example: Consider (K=5): Dataset is split into 5 folds: F1, F2, F3, F4, F5.

Iteration	Training Folds	Testing Folds
1	F2, F3, F4, F5	F1
2	F1, F3, F4, F5	F2

3	F1, F2, F4, F5	F3
4	F1, F2, F3, F5	F4
5	F1, F2, F3, F4	F5



Python implementation for k fold cross-validation

#Step 1: Import necessary libraries.

```
from sklearn.model_selection import cross_val_score, KFold
```

```
from sklearn.svm import SVC
```

```
from sklearn.datasets import load_iris
```

#Step 2: Load the dataset

```
iris = load_iris()
```

```
X, y = iris.data, iris.target
```

#Step 3: Create SVM classifier

SVC is a Support Vector Classification model from scikit-learn.

```
svm_classifier = SVC(kernel='linear')
```

Step 4: Define the number of folds for cross-validation

```
num_folds = 5
```

```
kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
```

Step 5: Perform k-fold cross-validation

```
cross_val_results = cross_val_score(svm_classifier, X, y, cv=kf)
```

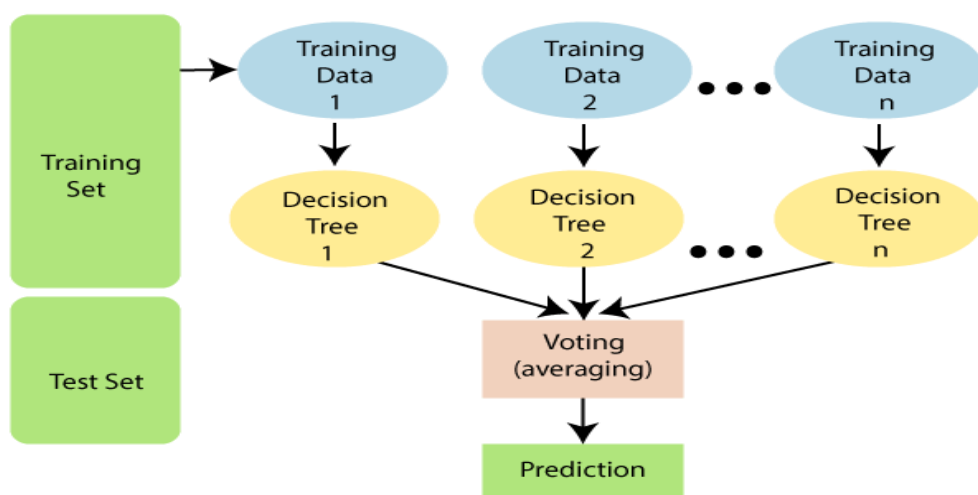
Step 6: Evaluation metrics

```
print(f'Cross-Validation Results (Accuracy): {cross_val_results}')
```

```
print(f'Mean Accuracy: {cross_val_results.mean()}')
```

4.3 Random Forests:

- Random Forest is a machine learning algorithm that uses many decision trees to make better predictions.
- Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression.
- This helps in improving accuracy and reducing errors. It can be used for both classification and regression problems in ML.
- It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.
- The below diagram explains the working of the Random Forest algorithm:



Random forest algorithm steps

Random Forest works in two-phase first is to create the random forest by combining N decision tree and second is to make predictions for each tree created in the first phase. The working process can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

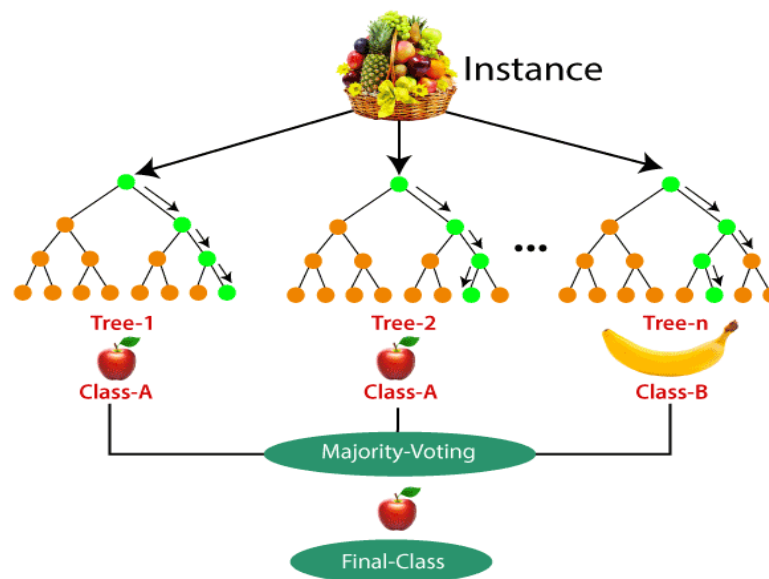
Step-2: Build the decision trees associated with the selected data points (Subsets).

Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

For example: Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the random forest classifier predicts the final decision. Consider the below image:



Applications of random forest

There are mainly four sectors where Random forest mostly used:

1. **Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
2. **Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
3. **Land Use:** We can identify the areas of similar land use by this algorithm.
4. **Marketing:** Marketing trends can be identified using this algorithm.

Key features of random forest

Some of the key features of random forest are discussed below:

1. **High predictive accuracy:** Imagine Random Forest as a team of decision-making wizards. Each wizard (decision tree) looks at a part of the problem, and together, they weave their insights into a powerful prediction tapestry. This teamwork often results in a more accurate model than what a single wizard could achieve.
2. **Resistance to overfitting:** Random Forest is like a cool-headed mentor guiding its apprentices (decision trees). Instead of letting each apprentice memorize every detail of their training, it encourages a more well-rounded understanding. This approach helps prevent getting too caught up with the training data which makes the model less prone to overfitting.
3. **Large datasets handling:** Dealing with a mountain of data? Random Forest tackles it like a seasoned explorer with a team of helpers (decision trees). Each helper takes on a part of the dataset, ensuring that the expedition is not only thorough but also surprisingly quick.

4. **Variable importance assessment:** Think of random forest as a detective at a crime scene, figuring out which clues (features) matter the most. It assesses the importance of each clue in solving the case, helping you focus on the key elements that drive predictions.
5. **Built-in cross-validation:** Random Forest is like having a personal coach that keeps you in check. As it trains each decision tree, it also sets aside a secret group of cases (out-of-bag) for testing. This built-in validation ensures your model doesn't just ace the training but also performs well on new challenges.
6. **Handling missing values:** Life is full of uncertainties, just like datasets with missing values. Random Forest is the friend who adapts to the situation, making predictions using the information available. It doesn't get flustered by missing pieces; instead, it focuses on what it can confidently tell us.
7. **Parallelization for speed:** Random Forest is your time-saving buddy. Picture each decision tree as a worker tackling a piece of a puzzle simultaneously. This parallel approach taps into the power of modern tech, making the whole process faster and more efficient for handling large-scale projects.

Advantages of random forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of random forest

- Although random forest can be used for both classification and regression tasks, it is not more suitable for regression tasks.

4.4 Boosting

Boosting is an ensemble modeling technique that attempts to build a strong classifier from the number of weak classifiers. It is done by building a model by using weak models in series. Firstly, a model is built from the training data. Then the second model is built which tries to correct the errors present in the first model. This procedure is continued and models are added until either the complete training data set is predicted correctly or the maximum number of models are added.

Advantages of boosting

- **Improved accuracy**- Boosting can improve the accuracy of the model by combining several weak models' accuracies and averaging them for regression or voting over them for classification to increase the accuracy of the final model.
- **Robustness to overfitting** – Boosting can reduce the risk of overfitting by reweighting the inputs that are classified wrongly.
- **Better handling of imbalanced data** – Boosting can handle the imbalance data by focusing more on the data points that are misclassified
- **Better interpretability** – Boosting can increase the interpretability of the model by breaking the model decision process into multiple processes.

Training of boosting model

1. Initialize the dataset and assign equal weight to each of the data point.
2. Provide this as input to the model and identify the wrongly classified data points.
3. Increase the weight of the wrongly classified data points.
4. if (got required results)
 Goto step 5

else

Goto step 2

5. End



Fig: Training a boosting model

The explanation for training the boosting model:

The above diagram explains the AdaBoost algorithm in a very simple way. Let's try to understand it in a stepwise process:

- **B1** consists of 10 data points which consist of two types namely plus(+) and minus(-) and 5 of which are plus(+) and the other 5 are minus(-) and each one has been assigned equal weight initially. The first model tries to classify the data points and generates a vertical separator line but it wrongly classifies 3 plus(+) as minus(-).
- **B2** consists of the 10 data points from the previous model in which the 3 wrongly classified plus(+) are weighted more so that the current model tries more to classify these pluses(+) correctly. This model generates a vertical

separator line that correctly classifies the previously wrongly classified pluses(+) but in this attempt, it wrongly classifies three minuses(-).

- **B3** consists of the 10 data points from the previous model in which the 3 wrongly classified minus(-) are weighted more so that the current model tries more to classify these minuses(-) correctly. This model generates a horizontal separator line that correctly classifies the previously wrongly classified minuses(-).
- **B4** combines together B1, B2, and B3 in order to build a strong prediction model which is much better than any individual model used.

Boosting vs Bagging

Boosting	Bagging
In Boosting we combine predictions that belong to different types .	Bagging is a method of combining the same type of prediction.
The main aim of boosting is to decrease bias, not variance	The main aim of bagging is to decrease variance not bias
At every successive layer Models are weighted according to their performance.	All the models have the same weightage
New Models are influenced by the accuracy of previous Models	All the models are independent of each other

Disadvantages of boosting algorithms

Boosting algorithms also have some disadvantages these are:

- Boosting algorithms are vulnerable to the outliers
- It is difficult to use boosting algorithms for real-time applications.
- It is computationally expensive for large datasets.

4.4 Hyper-parameter Tuning:

- **Hyper-parameter tuning** is the process of selecting the best values for a model's hyper-parameters to optimize performance.
- Hyper-parameters are settings that control the learning process of the model such as the learning rate, the number of neurons in a neural network or the kernel size in a support vector machine.
- The goal of hyper-parameter tuning is to find the values that lead to the best performance on a given task.

Hyper-parameters

- Hyper-parameters are adjustable parameters that control the learning process of a model but are not learned from the data.
- Hyper-parameters are often used to tune the performance of a model and they can have a significant impact on the model's accuracy, generalization and other metrics.
- Common hyper-parameters include:
- **Learning rate** – Controls how much a model updates weights at each step. Too high can cause instability, too low slows convergence.
- **Number of trees (for boosting algorithms)** – Determines how many weak learners contribute to the final ensemble.
- **Maximum depth** – Limits complexity to prevent overfitting in tree-based models.

- **Regularization terms (L1, L2)** – Helps prevent overfitting by controlling the magnitude of model weights.
- **Batch size (in deep learning)** – Defines how many samples are processed before updating the model.
- **Number of clusters (in k-modes clustering)** – Specifies how many groups should be formed in categorical data clustering.

Hyper-parameter tuning techniques

Models can have many hyper-parameters and finding the best combination of parameters can be treated as a search problem. The two best strategies for Hyper-parameter tuning are:

Grid Search:

- Grid search is a brute-force hyper-parameter tuning technique that systematically evaluates all possible combinations of specified hyper-parameters to find the optimal settings.
- It is particularly effective when the search space is relatively small and well-defined.

How grid search works:

- Define the hyper-parameters and their possible values.
- Train the model using each combination of hyper-parameters.
- Evaluate performance using a validation set.
- Select the combination that yields the best results.

Randomized Search - Randomized search is a hyper-parameter tuning technique that selects values at random from predefined ranges rather than systematically testing every combination, like Grid Search. This makes it more efficient for finding a good configuration in a reasonable amount of time.

How It Works:

- Define a range of hyper-parameters: You specify possible values for each hyper-parameter.
- Random sampling: Instead of testing all combinations, it picks values randomly.
- Train & evaluate models: Each sampled set is used to train a model, and performance is assessed using validation data.
- Repeat for a fixed number of iterations (n_iter): The process is repeated for a limited number of trials.
- Select the best configuration: The best-performing set of hyper-parameters is chosen based on validation metrics.

Key Features:

- **Random sampling:** Instead of testing all possibilities, it picks random points in the search space.
- **Computational efficiency:** Often finds well-performing hyper-parameters faster than Grid Search.
- **Supports large search spaces:** Works well when the number of hyper-parameters is high.